2322ENG: Engineering C

Programming Laboratory 7: Files and Project 2

Instructions:

This laboratory is designed to introduce you to files, as well as to revise material from previous weeks. There are assessable exercises at the end of the laboratory. Note that you will also need to work on the second stage of the project in this weeks lab session.

You should have read the first part of Topic 4 notes and watched all of the associated videos prior to attending this lab!

Homework Questions (for presentation):

These questions should be completed *prior* to the lab session. You may be called to present one of these questions in front of the class during the first half hour of the session, and this presentation will count for 5% of your final grade for the course. You will be called to do two such presentations during the trimester, at random times. Note that you will need to write and explain your solution on a whiteboard, not on a computer, so if you need to refer to code it should be on paper. Each question should take only a few minutes to complete, and you may be asked a few simple questions while presenting. If you are not present in the laboratory when you are due to present, you will receive 0 for this assessment item.

1. **What C function is used to open a file? Write C code that opens a file called "file.txt" in *read* mode, and stores the result in an appropriate variable.**
2. **Write a C program that opens a file called "data.txt" in write mode, then writes the integers 1 through 10 to the file, one number per line. The file should then be closed.**
3. **Write a C program to read the file created by Q2, and print out each number to the screen (one per line again).**
4. **Write a C *function* that takes a filename (string) as input, and returns the number of characters in that file. To determine this, simply continue to read characters from the file until it is at the end.**
5. **Repeat question 4, but use the file position functions (fseek and ftell) to determine the length of the file in bytes.**
6. **Write a C program that creates an array of 50 floating point numbers, and assigns them the values 0, 0.5, 1, 1.5, 2, …, 24.5. Next, write all of these numbers to a *binary* file called "floats.dat", using a single C statement to write the entire array at once. Note that the numbers must be written in binary form, not as text.**

File Pointers

File pointers are the essential element that makes file operations possible in C. You can think of them quite simply as a mechanism for accessing files – the technical details do not matter, as you will never need to delve into the internals of them. What do they do? Well, a file pointer refers to a chunk of memory that contains all of the OS specific information necessary to manipulate a file. Things like the position in the file system where the file resides (usually an inode or FAT entry), the size of the file, the mode it's been opened in, and so on. All the file operations have access to the file pointer,

and thus have access to all of this information, allowing them to work properly. We as programmers don't need to know any of it, since the file functions will do all the work for us.

A file pointer is a created data type. It is not native to the C language like int, char, float, and double are, rather it was added to the language by a library (stdio in this case). However now that it has been added, we can use file pointers as if they were a native data type, and declare them just like any other variable. Since they are pointers, you need to put a * symbol before each variable when declaring them, however usually not when using them. To declare a file pointer, use the syntax shown below:

```
#include <stdio.h>
int main ( void ){

     FILE *fp ;

}
```

Like other variables, a file pointer is undefined when declared. Attempting to use a file pointer before it has been assigned a 'real' value will cause an error, most likely a segfault causing your program to instantly crash. So, how can you assign them a value? We do this by *opening* a file. This is done with the *fopen* function, which takes a filename and file mode as inputs, and returns a file pointer if the file was successfully opened. If the file could not be opened for some reason (it doesn't exist, you don't have rights to access it, etc), NULL is returned instead. It is important that you check for a NULL return value before attempting to use the file pointer, otherwise your program will crash. Each file pointer can only access one file *at a time*, so if you need to open two files at the same time, you'll need two file pointers. However, once a file has been closed, the same file pointer variable can be used to open a new file (or the same file again).

fopen requires two inputs. The first is a string representing the filename (and path, if it is not in the same directory as the program is running). The second is the *mode* in which the file should be opened. This will be 1-3 characters, the first telling the function whether the file will be read of written to (or both), and the second is optional, indicating whether the file should be opened in ASCII mode (no character) or binary mode ("b"). For example:

```
#include <stdio.h>
int main ( void ){

     FILE *fp1, *fp2 ;

     fp1 = fopen ( "file1", "r" ) ;
     // file1 opened in read mode, as an ascii file

     fp2 = fopen ( "file2", "wb" ) ;
     // file 2 opened in write mode as a binary file

}
```

The code shown above does *not* check for NULL file pointers, and so is rather dangerous. You should always perform this check, so your code might look like this instead:

```
#include <stdio.h>
int main ( void ){

     FILE *fp1, *fp2 ;

     if ((fp1 = fopen ( "file1", "r" ))==NULL){
          printf ( "Cannot open file!\n" ) ;
          exit ( -1 ) ;
     // file1 opened in read mode, as an ascii file

     if ((fp2 = fopen ( "file2", "wb" ))==NULL){
          printf ( "Cannot open file!\n" ) ;
          exit ( -1 ) ;
     // file 2 opened in write mode as a binary file

}
```

This code now opens the file then checks if the resulting file pointer is NULL. If so, the program is terminated with the *exit* command. If both files were opened successfully, the program continues and the file pointers can now be used to read and write to file1 and file2 respectively.

File Operations

The C functions used to read and write to files are very similar to those you've seen already for normal IO. In most cases, the only difference is an 'f' in the function name, and one additional argument indicating the file to be used. The following table shows some commonly used file functions:

Normal IO Function	**File Version**
printf	*fprintf*
scanf	*fscanf*
getc	*fgetc*
putc	*fputc*
gets	*fgets*
puts	*fputs*

Look at the man page for each of these functions and you will see that there is an extra argument for each one – a file pointer. Obviously, it only makes sense to use *fprintf* and other functions that write to a file with files that have been opened for writing, and *fgets* and other reading functions with files that are in read mode.

All of these functions operate in exactly the same way as their normal equivalents. They assume that the file is a *stream* of information, and process this stream one character (byte) at a time. When you read from the file, the position in the file is updated to indicate that those characters have been 'read', and subsequent reads will get the next byte in the file instead.

End of File

Unlike the keyboard, input from a file will eventually end. It is very important that you recognise when this has happened so you can stop trying to read data. There are two ways to do this. Firstly, text-based files will use the EOF character to indicate that the file is finished. This will be the return

value of the fscanf function if the file is at the end. However this will only happen if fscanf is unable to read *any* of the symbols. If it reads even 1 item correctly, then reaches the end of the file, this will not be considered the end of file, and 1 will be returned.

For binary files, you cannot use the EOF character to check for end of file, since this character could appear in the file by chance. Instead, there is a function called *feof* that can be used instead. This function takes a file pointer as input, and returns 1 if the file is ended, or 0 otherwise. Note that this can also be used for text files.

When writing to files, there's no end of file indicator, continue writing until you're finished then just close the file.

Closing Files

Once you are finished with a file, you should close it. To do this, use the *fclose* function, which takes a file pointer as its only argument. Once you have closed the file, the file pointer will no longer be valid, and should not be used until you have *fopen*'ed another file.

Assessable Question

You may be called to present one of these questions during the last hour of the session, and this presentation will count for 5% of your final grade for the course. You will be called to do two such presentations during the trimester, at random times. Note that you will need to explain your solution to the demonstrator and make changes to the code if requested to demonstrate your understanding. Each question should take less than 5 minutes to complete, and you may be asked a few simple questions while presenting. If you are not present in the laboratory when you are due to present, you will receive 0 for this assessment item.

1. **Write a C program that reads a file called "floats.txt" that contains two floating point numbers per line, and then prints those numbers to the screen separated by a tab space. Create a test file to run the program to ensure it works. Note that the best approach is to read each line of the file first, then read the floats from that string rather than reading the floats directly from the file.**

2. **A small grocery store uses a very basic file format to store prices for its goods. This file format has one item per line. The first word of the line is the name of the product, and after this is the price of the product in dollars (no dollar sign). For example, a typical file might look like:**

   bread 2.50
   milk 1.90
   pizza 10.95
   coke 99.95

   Write a C program which reads one such file (create your own for testing) into *two arrays*, one called "items" which stores the product names, and the other called "prices" which stores all the prices. When creating the arrays, assume that the maximum length of a product name should be capped at 100 characters, and that there is a maximum of

1000 products in the file. Once all values are stored in the array, print the product list to the screen, including dollar signs in the prices as necessary along with headers for each column, for example:

```
ITEM PRICE
==============
bread  $2.50
milk   $1.90
pizza  $10.95
coke   $99.95
```

(1 mark)

3. **Write a C program that reads integers from the user, and then writes those integers to a text file called "numbers.txt". The numbers should be stored in text format, one per line. Your program should stop when the number 0 is entered (this should not be written to the file).**

4. **Work on milestone 2 of the project (see program description).**